

APLICACIONES DISTRIBUIDAS - SOFT-R - A - [7MO NIVEL] | CORTE 1

Tema: EV - Unidad I - Aporte Parte Práctica

Nombres y Apellidos: Pedro Leonardo Castro López

Dede desarrollar la solución del problema planteado en el documento adjunto.
Revise la rúbrica de evaluación.

RegistroDistribuido

Paso 1 — Definición del modelo de mensaje (Message.java)

Lo primero que hice fue definir cómo se iban a comunicar los nodos entre sí y con el cliente. Para eso creé la clase Message.java, que es básicamente el formato estándar de todos los mensajes que viajan por la red en el sistema.

```
package ec.edu.uteg.distribuidas.model;

import com.fasterxml.jackson.annotation.JsonCreator;
import com.fasterxml.jackson.annotation.JsonProperty;

public class Message {

    public enum Type {
        OPERATION,
        HEARTBEAT,
        HEARTBEAT_ACK,
        ELECTION,
        ELECTION_OK,
        COORDINATOR,
        SYNC_CLOCK,
        AUTH
    }

    private final Type type;
    private final int senderId;
    private final long lamportClock;
    private final String payload;
    private final String token;

    @JsonCreator
    public Message(
        @JsonProperty("type") Type type,
        @JsonProperty("senderId") int senderId,
        @JsonProperty("lamportClock") long lamportClock,
        @JsonProperty("payload") String payload,
        @JsonProperty("token") String token) {
        this.type = type;
        this.senderId = senderId;
        this.lamportClock = lamportClock;
        this.payload = payload;
        this.token = token;
    }

    public Type getType() { return type; }
```

```

public int getSenderId() { return senderId; }
public long getLamportClock() { return lamportClock; }
public String getPayload() { return payload; }
public String getToken() { return token; }

@Override
public String toString() {
    return "Message{type=" + type + ", sender=" + senderId + ",
clock=" + lamportClock + ", payload='" + payload + "'}";
}
}

```

Esta clase tiene los siguientes campos:

- type — el tipo de mensaje, puede ser OPERATION, HEARTBEAT, ELECTION, COORDINATOR, entre otros
- senderId — el ID del nodo que envía el mensaje
- lamportClock — el valor del reloj lógico en el momento del envío
- payload — el contenido del mensaje, por ejemplo el texto de la operación
- token — el token de autenticación del cliente

Paso 2 - Definición del modelo de evento de log (LogEvent.java)

Después de definir cómo se comunican los nodos, necesitaba una forma de registrar y ordenar todos los eventos que ocurren en el sistema. Para eso creé la clase LogEvent.java.

Esta clase representa un evento que ocurrió en algún nodo, y tiene estos campos:

- lamportClock — el valor del reloj de Lamport en el momento del evento
- nodeId — qué nodo generó el evento
- description — descripción de lo que pasó
- wallTime — el tiempo real del sistema cuando ocurrió

```

• @Override
public int compareTo(LogEvent other) {
    if (this.lamportClock != other.lamportClock) {
        return Long.compare(this.lamportClock,
other.lamportClock);
    }
    return Integer.compare(this.nodeId, other.nodeId);
}

```

Paso 3 — Configuración del nodo (NodeConfig.java)

Con el modelo de mensajes y el log ya definidos, el siguiente paso fue configurar cómo se iba a identificar cada nodo en el sistema. Para eso creé la clase NodeConfig.java.

Esta clase lee automáticamente los valores del archivo application.properties de cada nodo gracias a la anotación @ConfigurationProperties. Los campos que maneja son:

- id — el número identificador del nodo (1, 2 o 3)
- port — el puerto TCP donde escucha el nodo
- peers — la lista de los otros nodos con su host y puerto
- validToken — el token que deben presentar los clientes para autenticarse
- heartbeatIntervalMs — cada cuánto tiempo envía latidos (2000ms)
- heartbeatTimeoutMs — cuánto tiempo espera antes de marcar un nodo como caído (6000ms)

```
• private int id;  
  private int port;  
  private List<Map<String, Object>> peers;  
  private String validToken;  
  private int heartbeatIntervalMs = 2000;  
  private int heartbeatTimeoutMs = 6000;
```

así quedó el archivo de configuración para el nodo 1 por ejemplo:

```
spring.application.name=RegistroDistribuido-N1  
node.id=1  
node.port=8081  
node.valid-token=TOKEN-SECRETO-2025  
node.heartbeat-interval-ms=2000  
node.heartbeat-timeout-ms=6000  
node.peers[0].id=2  
node.peers[0].host=localhost  
node.peers[0].port=8082  
node.peers[1].id=3  
node.peers[1].host=localhost  
node.peers[1].port=8083  
spring.main.web-application-type=none
```

así quedó el archivo de configuración para el nodo 2 por ejemplo:

```
spring.application.name=RegistroDistribuido-N2  
node.id=2  
node.port=8082  
node.valid-token=TOKEN-SECRETO-2025  
node.heartbeat-interval-ms=2000  
node.heartbeat-timeout-ms=6000  
node.peers[0].id=1  
node.peers[0].host=localhost  
node.peers[0].port=8081  
node.peers[1].id=3  
node.peers[1].host=localhost
```

```
node.peers[1].port=8083
spring.main.web-application-type=none
```

así quedó el archivo de configuración para el nodo 3 por ejemplo:

```
spring.application.name=RegistroDistribuido-N3
node.id=3
node.port=8083
node.valid-token=TOKEN-SECRETO-2025
node.heartbeat-interval-ms=2000
node.heartbeat-timeout-ms=6000
node.peers[0].id=1
node.peers[0].host=localhost
node.peers[0].port=8081
node.peers[1].id=2
node.peers[1].host=localhost
node.peers[1].port=8082
spring.main.web-application-type=none
```

se implemento cualquier lógica porque todos los demás componentes necesitan saber en qué puerto escuchar, quiénes son sus vecinos y cuál es el token válido.

Paso 4 — Implementación del reloj de Lamport (LamportClock.java)

Con la configuración lista, el siguiente paso fue implementar el reloj lógico de Lamport. Este es uno de los componentes más importantes del sistema porque es el que permite ordenar los eventos de forma consistente entre los 3 nodos sin necesidad de un reloj físico global.

Implementé las 3 reglas de Lamport:

- **Regla 1** — antes de cualquier evento local, el contador se incrementa en 1
- **Regla 2** — al enviar un mensaje, se adjunta el valor actual del reloj
- **Regla 3** — al recibir un mensaje, el reloj se actualiza con $\max(\text{local}, \text{recibido}) + 1$

```
• package ec.edu.uteq.distribuidas.node;

import org.springframework.stereotype.Component;
import java.util.concurrent.atomic.AtomicLong;

@Component
public class LamportClock {

    private final AtomicLong clock = new AtomicLong(0);

    public long tick() {
        return clock.incrementAndGet();
    }
}
```

```

    public long update(long received) {
        long updated;
        long current;
        do {
            current = clock.get();
            updated = Math.max(current, received) + 1;
        } while (!clock.compareAndSet(current, updated));
        return updated;
    }

    public long get() {
        return clock.get();
    }
}

```

Usé AtomicLong para que el reloj sea thread-safe, es decir, que funcione correctamente aunque varios hilos lo actualicen al mismo tiempo. El método tick() se usa para eventos locales y el método update() se usa cada vez que llega un mensaje de otro nodo.

Paso 5 — Registro de eventos (EventLog.java)

Con el reloj de Lamport funcionando, el siguiente paso fue crear el componente que almacena y ordena todos los eventos que ocurren en cada nodo. Para eso creé la clase EventLog.java.

Esta clase tiene una lista donde se van guardando todos los eventos, y dos métodos principales:

- add() — agrega un nuevo evento a la lista
- getSorted() — devuelve todos los eventos ordenados usando el compareTo que definí en LogEvent.java
- print() — imprime el log ordenado en consola para que se pueda ver durante la demostración

```

• package ec.edu.uteq.distribuidas.node;

import ec.edu.uteq.distribuidas.model.LogEvent;
import org.springframework.stereotype.Component;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

@Component
public class EventLog {

    private final CopyOnWriteArrayList<LogEvent> events = new
CopyOnWriteArrayList<>();

    public void add(LogEvent event) {
        events.add(event);
    }
}

```

```

    }

    public List<LogEvent> getSorted() {
        List<LogEvent> sorted = new ArrayList<>(events);
        Collections.sort(sorted);
        return sorted;
    }

    public void print(int nodeId) {
        System.out.println("\n=== LOG ORDENADO - Nodo " + nodeId
+ " ===");
        getSorted().forEach(System.out::println);

        System.out.println("=====\\
n");
    }
}

```

Paso 6 — Cliente TCP entre nodos (NodeTcpClient.java)

Con el log y el reloj listos, el siguiente paso fue implementar la comunicación entre nodos. Para eso creé la clase NodeTcpClient.java, que es el componente que usa cada nodo para enviarle mensajes a sus peers.

Lo más importante de esta clase es el **framing de mensajes**. Este es el mecanismo que garantiza que dos mensajes enviados seguidos no se solapen ni se trunquen en el receptor. Lo implementé así:

- Primero se envían **4 bytes** que indican el tamaño del mensaje
- Luego se envía el **contenido JSON** del mensaje

```

public Message send(Message message) throws IOException {
    try (Socket socket = new Socket(host, port);
        DataOutputStream out = new
DataOutputStream(socket.getOutputStream());
        DataInputStream in = new
DataInputStream(socket.getInputStream())) {

        byte[] data = mapper.writeValueAsBytes(message);
        out.writeInt(data.length);
        out.write(data);
        out.flush();

        int len = in.readInt();
        byte[] response = new byte[len];
        in.readFully(response);
        return mapper.readValue(response, Message.class);
    }
}

```

También implementé un segundo método sendNoReply() para los casos donde no se necesita esperar respuesta, como los heartbeats y la replicación de operaciones:

```

public void sendNoReply(Message message) {
    try (Socket socket = new Socket(host, port);
        DataOutputStream out = new

```

```

DataOutputStream(socket.getOutputStream())) {
    socket.setSoTimeout(1000);
    byte[] data = mapper.writeValueAsBytes(message);
    out.writeInt(data.length);
    out.write(data);
    out.flush();
} catch (IOException ignored) {}
}

```

Paso 7 — Manejador de conexiones (ConnectionHandler.java)

Creé esta clase para procesar cada mensaje que llega al nodo. Implementa Runnable para que cada conexión se maneje en un hilo separado y el nodo pueda atender varios clientes al mismo tiempo.

El proceso es simple: lee los 4 bytes de longitud, lee el JSON, lo pasa al NodeService y si hay respuesta la devuelve con el mismo framing:

```

package ec.edu.uteq.distribuidas.node;

import com.fasterxml.jackson.databind.ObjectMapper;
import ec.edu.uteq.distribuidas.model.Message;

import java.io.*;
import java.net.Socket;

public class ConnectionHandler implements Runnable {

    private final Socket socket;
    private final NodeService nodeService;
    private final ObjectMapper mapper = new ObjectMapper();

    public ConnectionHandler(Socket socket, NodeService nodeService) {
        this.socket = socket;
        this.nodeService = nodeService;
    }

    @Override
    public void run() {
        try (DataInputStream in = new
DataInputStream(socket.getInputStream());
            DataOutputStream out = new
DataOutputStream(socket.getOutputStream())) {

            int len = in.readInt();
            byte[] data = new byte[len];
            in.readFully(data);

            Message incoming = mapper.readValue(data, Message.class);
            Message response = nodeService.handle(incoming);

            if (response != null) {
                byte[] respData = mapper.writeValueAsBytes(response);
                out.writeInt(respData.length);
                out.write(respData);
                out.flush();
            }
        }
    }
}

```

```

        } catch (IOException e) {

        } finally {
            try { socket.close(); } catch (IOException ignored) {}
        }
    }
}

```

Paso 8 — Servidor TCP (TcpServer.java)

Con el manejador de conexiones listo, el siguiente paso fue crear el servidor que escucha en el puerto del nodo y acepta las conexiones entrantes. Para eso creé la clase `TcpServer.java`.

Esta clase abre un `ServerSocket` en el puerto configurado y por cada conexión que llega crea un `ConnectionHandler` en un hilo separado del pool:

```

package ec.edu.uteq.distribuidas.node;

import ec.edu.uteq.distribuidas.config.NodeConfig;
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

@Component
public class TcpServer {

    private final NodeConfig config;
    private final NodeService nodeService;
    private ServerSocket serverSocket;
    private volatile boolean running = true;
    private final ExecutorService pool =
        Executors.newCachedThreadPool();

    public TcpServer(NodeConfig config, NodeService nodeService) {
        this.config = config;
        this.nodeService = nodeService;
    }

    @PostConstruct
    public void start() {
        Thread serverThread = new Thread(() -> {
            try {
                serverSocket = new ServerSocket(config.getPort());
                System.out.println("[N" + config.getId() + "] TCP
escuchando en puerto " + config.getPort());
                while (running) {
                    try {
                        Socket client = serverSocket.accept();
                        pool.submit(new ConnectionHandler(client,
nodeService));

```

```

        } catch (IOException e) {
            if (running) e.printStackTrace();
        }
    }
} catch (IOException e) {
    System.err.println("[N" + config.getId() + "] Error al
abrir puerto " + config.getPort() + ": " + e.getMessage());
}
}, "tcp-server-n" + config.getId());
serverThread.setDaemon(true);
serverThread.start();
}

@PreDestroy
public void stop() {
    running = false;
    try {
        if (serverSocket != null) serverSocket.close();
    } catch (IOException ignored) {}
    pool.shutdownNow();
}
}

```

`Executors.newCachedThreadPool()` para el pool de hilos, lo que significa que crea un hilo nuevo por cada conexión pero los reutiliza si están disponibles. También implementé el método `stop()` con `@PreDestroy` para cerrar el servidor limpiamente cuando el nodo se apaga.

Paso 9 — Servicio principal del nodo (NodeService.java)

Este es el componente más importante de todo el sistema porque contiene toda la lógica de negocio. Aquí se juntan todas las partes anteriores — el reloj de Lamport, el log de eventos, los heartbeats, el algoritmo Bully y la autenticación.

Al iniciar el nodo, el método `init()` hace 3 cosas automáticamente:

```

public void init() {

    for (NodeConfig.PeerInfo p : config.getPeerList()) {
        peers.put(p.id(), new NodeTcpClient(p.host(), p.port()));
        lastHeartbeat.put(p.id(), System.currentTimeMillis());
    }

    log("Nodo N" + config.getId() + " iniciado en puerto " +
config.getPort());

    scheduler.scheduleAtFixedRate(this::sendHeartbeats, 2000,
config.getHeartbeatIntervalMs(), TimeUnit.MILLISECONDS);

    scheduler.scheduleAtFixedRate(this::checkHeartbeats, 3000,
config.getHeartbeatIntervalMs(), TimeUnit.MILLISECONDS);
}

```

```

        scheduler.schedule(this::startElection, 4000,
            TimeUnit.MILLISECONDS);
    }

```

Manejo de mensajes — Parte A

Todos los mensajes entrantes pasan por el método `handle()` que los distribuye según su tipo:

```

public Message handle(Message msg) {
    switch (msg.getType()) {
        case OPERATION -> { return handleOperation(msg); }
        case HEARTBEAT -> { return handleHeartbeat(msg); }
        case HEARTBEAT_ACK -> { handleHeartbeatAck(msg); return null; }
    }

    case ELECTION -> { return handleElection(msg); }
    case ELECTION_OK -> { handleElectionOk(msg); return null; }
    case COORDINATOR -> { handleCoordinator(msg); return null; }
    default -> { return buildResponse(Message.Type.OPERATION,
        "UNKNOWN_TYPE"); }
    }
}

```

Paso 10 — Cliente interactivo (ClienteRegistro.java)

Creé este archivo para probar el sistema. Se conecta a un nodo y permite enviar operaciones desde la consola. Tiene dos comandos: `op` con token válido y `op-bad` con token inválido para probar la seguridad.

```

        String token = cmd.equals("op-bad") ? TOKEN_INVALIDO :
            TOKEN_VALIDO;

        Message msg = new Message(Message.Type.OPERATION, 0, 0, payload,
            token);

        try {
            Message response = sendAndReceive(host, port, msg);
            System.out.println("[RESPUESTA] " + response.getPayload());
        } catch (Exception e) {
            System.err.println("[ERROR] No se pudo conectar con " + host +
                ":" + port + " - " + e.getMessage());
        }
    }
}

```

Paso 11 — Configuración de los 3 nodos (application.properties)

Creé un archivo por cada nodo con su ID, puerto y la lista de sus peers:

```

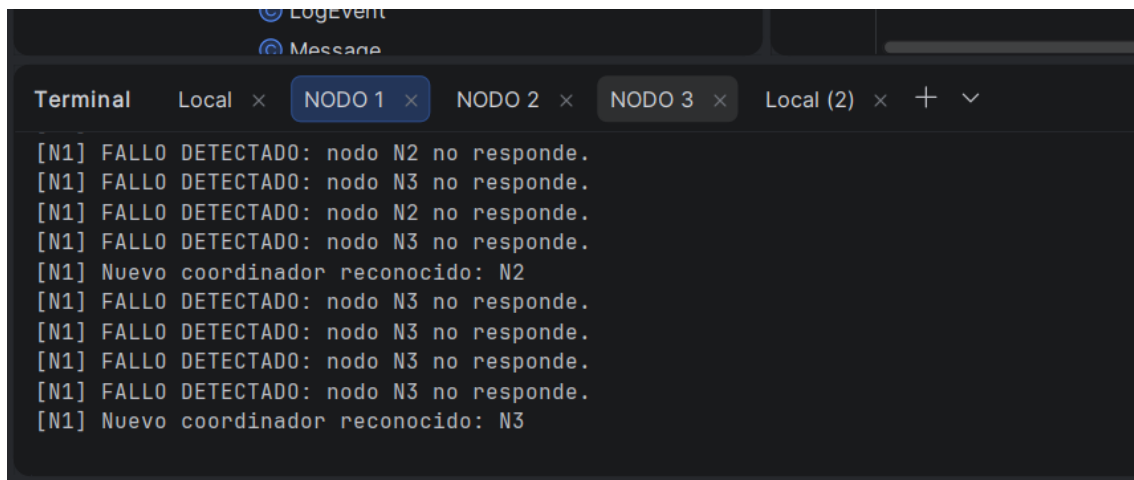
spring.application.name=RegistroDistribuido-N3
node.id=3
node.port=8083
node.valid-token=TOKEN-SECRETO-2025
node.heartbeat-interval-ms=2000
node.heartbeat-timeout-ms=6000

```

```
node.peers[0].id=1
node.peers[0].host=localhost
node.peers[0].port=8081
node.peers[1].id=2
node.peers[1].host=localhost
node.peers[1].port=8082
spring.main.web-application-type=none
```

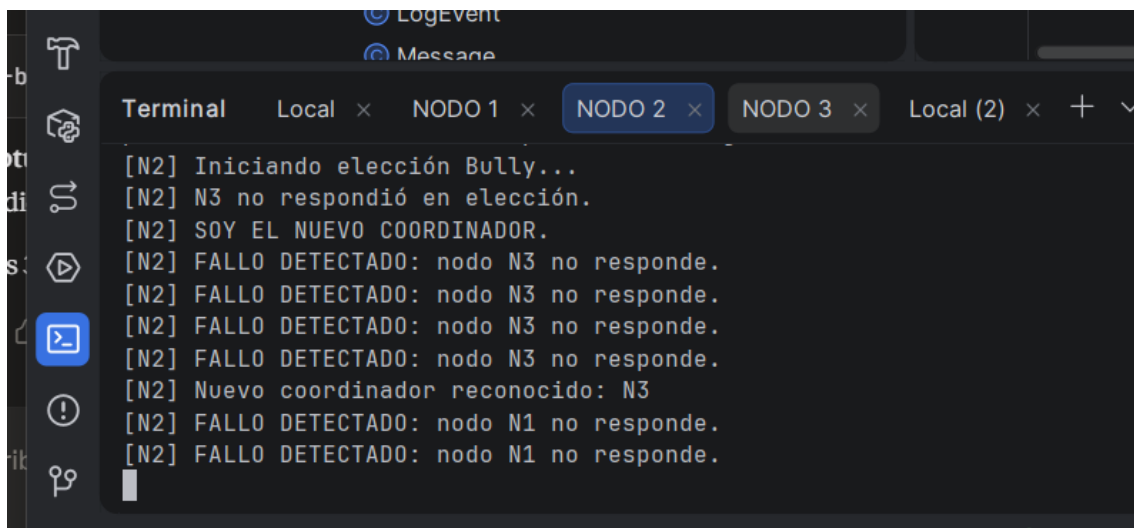
ANEXOS

op MOVIMIENTO-CARGA-001 NODO 1



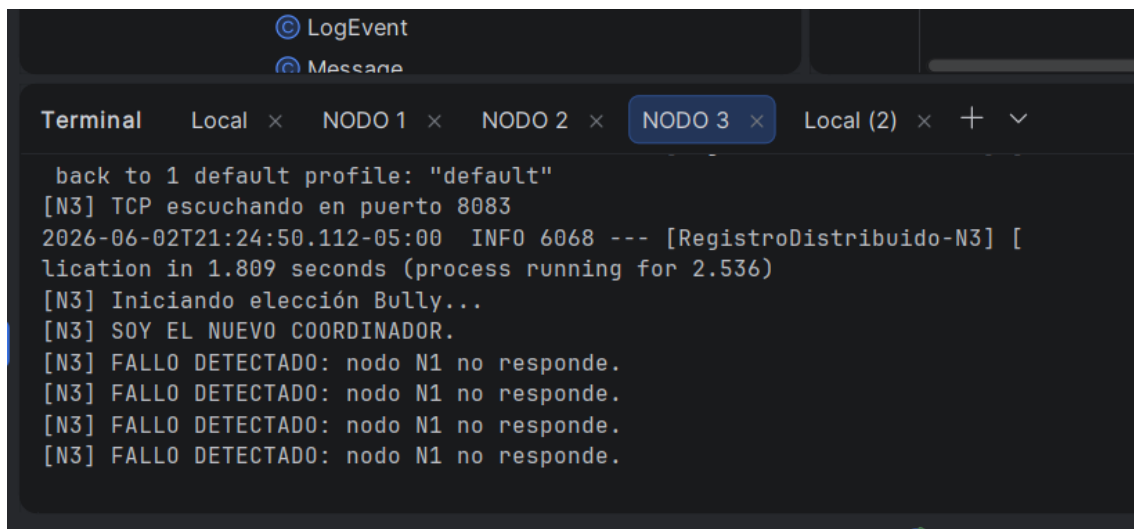
```
LogEvent
Message
Terminal Local x NODO 1 x NODO 2 x NODO 3 x Local (2) x + v
[N1] FALLO DETECTADO: nodo N2 no responde.
[N1] FALLO DETECTADO: nodo N3 no responde.
[N1] FALLO DETECTADO: nodo N2 no responde.
[N1] FALLO DETECTADO: nodo N3 no responde.
[N1] Nuevo coordinador reconocido: N2
[N1] FALLO DETECTADO: nodo N3 no responde.
[N1] FALLO DETECTADO: nodo N3 no responde.
[N1] FALLO DETECTADO: nodo N3 no responde.
[N1] FALLO DETECTADO: nodo N3 no responde.
[N1] Nuevo coordinador reconocido: N3
```

NODO 2



```
LogEvent
Message
Terminal Local x NODO 1 x NODO 2 x NODO 3 x Local (2) x + v
[N2] Iniciando elección Bully...
[N2] N3 no respondió en elección.
[N2] SOY EL NUEVO COORDINADOR.
[N2] FALLO DETECTADO: nodo N3 no responde.
[N2] FALLO DETECTADO: nodo N3 no responde.
[N2] FALLO DETECTADO: nodo N3 no responde.
[N2] FALLO DETECTADO: nodo N3 no responde.
[N2] Nuevo coordinador reconocido: N3
[N2] FALLO DETECTADO: nodo N1 no responde.
[N2] FALLO DETECTADO: nodo N1 no responde.
```

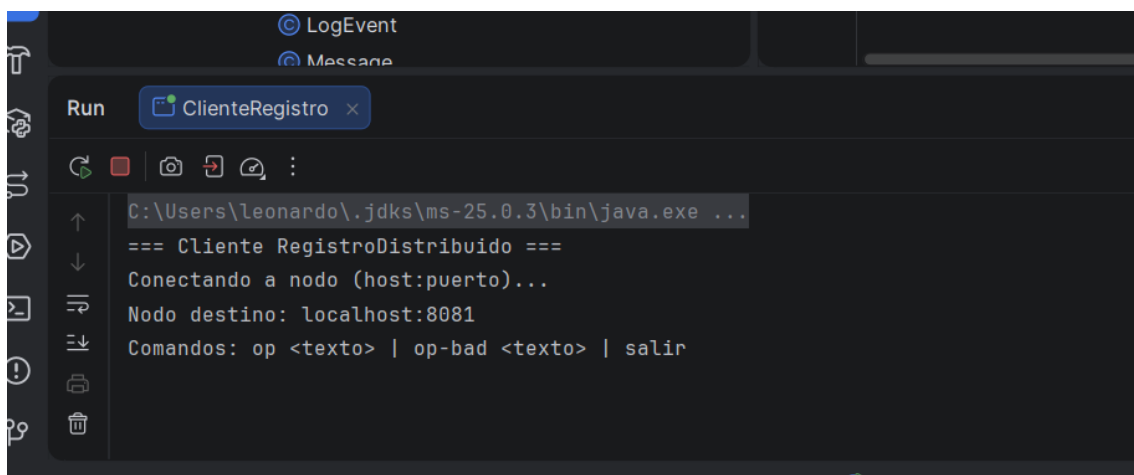
NODO 3



The screenshot shows a terminal window with a tab labeled 'NODO 3'. The terminal output displays the following log messages:

```
back to 1 default profile: "default"
[N3] TCP escuchando en puerto 8083
2026-06-02T21:24:50.112-05:00 INFO 6068 --- [RegistroDistribuido-N3] [
lication in 1.809 seconds (process running for 2.536)
[N3] Iniciando elección Bully...
[N3] SOY EL NUEVO COORDINADOR.
[N3] FALLO DETECTADO: nodo N1 no responde.
[N3] FALLO DETECTADO: nodo N1 no responde.
[N3] FALLO DETECTADO: nodo N1 no responde.
[N3] FALLO DETECTADO: nodo N1 no responde.
```

CLIENTE



The screenshot shows a terminal window with a tab labeled 'ClienteRegistro'. The terminal output displays the following log messages:

```
=== Cliente RegistroDistribuido ===
Conectando a nodo (host:puerto)...
Nodo destino: localhost:8081
Comandos: op <texto> | op-bad <texto> | salir
```